Centro Universitário de Excelência
Sistemas de Informação

MECANIQA — PERSISTÊNCIA COM JDBC E MYSQL

Autores: Rodrigo dos Anjos Lamartine
Lorran Edley Matos Ribeiro
Kauan Henrique

Vitória da Conquista, 2026

Agenda

O objetivo dessa apresentação é demonstrar a evolução da API MecaniQA para persistência em MySQL com JDBC puro, Mappers, exceções customizadas e camada Service.

1. Item: Contexto
Dados em memória evoluem para armazenamento definitivo em banco.

2. Item: Objetivo
JDBC puro, Mappers de ResultSet, exceções customizadas e Service.

3. Item: Banco de Dados
MySQL com script DDL/DML versionado e cinco tabelas.

4. Item: Arquitetura
Controller, Service e Repository genérico sobre JDBC.

5. Item: Resultados
Endpoints com status HTTP tratados manualmente nos controllers.

Contexto

Na etapa anterior, os dados ficavam em listas na memória:

- Ao reiniciar a API, todos os dados eram perdidos
- Repositórios em singletons, acessados direto pelos controllers

Agora a MecaniQA precisa de persistência definitiva e de uma arquitetura mais organizada.

Objetivo

Evoluir a API com os requisitos desta sprint:

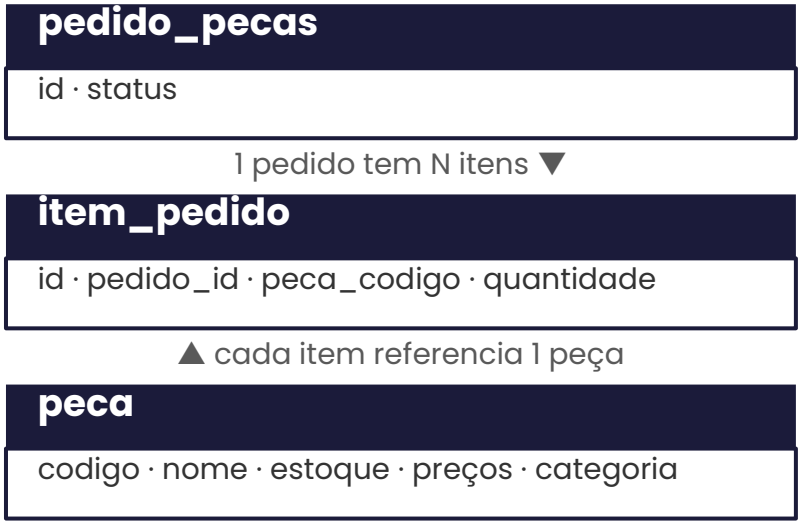
- Persistência no banco de dados MySQL
- JDBC nativo, com queries SQL escritas à mão
- Mappers para converter ResultSet em objetos
- Exceções customizadas, sem @ExceptionHandler
- Camada Service entre Controller e Repository

Restrição: nenhum ORM, JPA ou Hibernate.

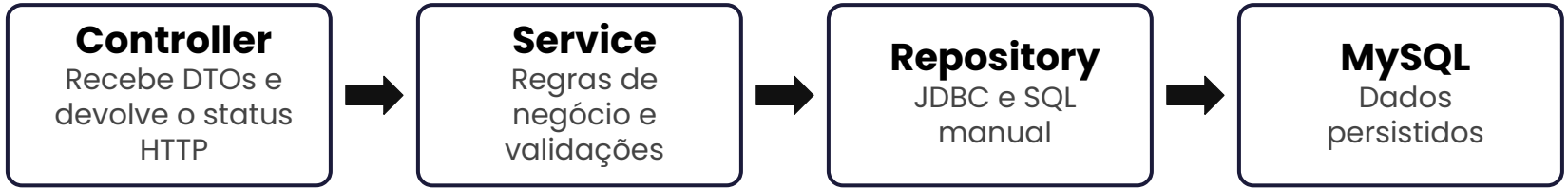
Banco de Dados

- MySQL como banco oficial
- Script database/mecaniqa.sql versionado no projeto
- DDL: 5 tabelas com chaves e restrições
- DML: dados iniciais de exemplo

Tabelas: peca, servico, ordem_servico, pedido_pecas e item_pedido.



Arquitetura em Camadas



- Controllers não chamam os repositórios diretamente
- Todos os repositórios implementam Repository<T, ID>

Repository Genérico

Contrato único de CRUD:

- save, findById, findAll
- update e deleteById

Implementado por PecaRepository, ServicoRepository, OrdemServicoRepository e PedidoPecasRepository.

```java
public interface Repository<T, ID> {

    T save(T entity);

    Optional<T> findById(ID id);

    List<T> findAll();

    boolean update(ID id, T entity);

    boolean deleteById(ID id);
}
```

Persistência com JDBC

- Queries SQL escritas à mão
- PreparedStatement evita SQL injection
- try-with-resources fecha a conexão
- SQLException vira PersistenciaException

```
public Optional<Peca> findById(String codigo) {
  try (Connection conn =
           connectionFactory.getConnection();
       PreparedStatement ps =
           conn.prepareStatement(SELECT_BY_ID)) {
    ps.setString(1, codigo);
    try (ResultSet rs = ps.executeQuery()) {
      return rs.next()
          ? Optional.of(mapper.map(rs))
          : Optional.empty();
    }
  } catch (SQLException e) {
    throw new PersistenciaException(
        "Erro ao buscar a peça.", e);
  }
}
```

Mappers de ResultSet

- Um Mapper por entidade
- Converte a linha do ResultSet em objeto de modelo
- Isola a leitura das colunas dos repositórios

```
public interface ResultSetMapper<T> {
  T map(ResultSet rs) throws SQLException;
}

public class OrdemServicoResultSetMapper
    implements ResultSetMapper<OrdemServico> {

  public OrdemServico map(ResultSet rs)
      throws SQLException {
    return new OrdemServico(
        rs.getLong("id"),
        rs.getString("descricao"),
        rs.getString("nome_cliente"),
        StatusOrdemServico.valueOf(
            rs.getString("status")));
  }
}
```

Exceções e Camada Service

Status HTTP definidos nos controllers:

- DadosInvalidosException → 400
- PecaNaoEncontradaException → 404
- RegraDeNegocioException → 409
- PersistenciaException → 500

O Service valida os dados e calcula o valor total do pedido.

```
@GetMapping("/{codigo}")
public ResponseEntity<?> buscarPorCodigo(
    @PathVariable String codigo) {
  try {
    return ResponseEntity.ok(
        service.buscarPorCodigo(codigo));
  } catch (RecursoNaoEncontradoException e) {
    return RespostaErro.de(
        HttpStatus.NOT_FOUND, e.getMessage());
  } catch (PersistenciaException e) {
    return RespostaErro.de(
        HttpStatus.INTERNAL_SERVER_ERROR,
        e.getMessage());
  }
}
```

Resultados

- Endpoints implementados para peças, serviços, ordens de serviço e pedidos de peças

Método	Rota	Retornos
POST	/api/pecas	201 ou 400
GET	/api/pecas/{codigo}	200 ou 404
PUT	/api/servicos/{codigo}	200, 400 ou 404
DELETE	/api/pecas/{codigo}	204, 404 ou 409
POST	/api/pedidos-pecas/{id}/itens	200, 400, 404 ou 409
PATCH	/api/ordens-servico/{id}/status	200, 400 ou 404

Considerações finais

A MecaniQA passou a persistir seus dados em MySQL usando apenas JDBC, com repositórios padronizados pela interface Repository, Mappers de ResultSet, exceções customizadas e uma camada Service. A separação em camadas facilita a manutenção e a evolução da API nas próximas sprints.

Referências

- Oracle. JDBC Basics — docs.oracle.com/javase/tutorial/jdbc
- MySQL Connector/J — dev.mysql.com/doc/connector-j
- MySQL Reference Manual — dev.mysql.com/doc/refman
- Spring Boot. Documentação oficial — spring.io/projects/spring-boot
- Documentação Java SE 21 — docs.oracle.com/en/java/javase/21